efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

CHAPITRE 4 : Interrogation d'une BD en algèbre relationnelle et en SQL

TI404 – Bases de données 1 : Concepts de base

Coordinatrice du cours : Lena TREBAUL
lena.trebaul@efrei.fr

Liens entre algèbre relationnelle et SQL

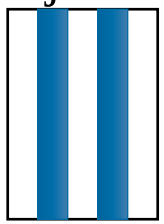
- Le modèle relationnel donne un ensemble d'opérations pour manipuler des relations avec les opérateurs de l'**algèbre relationnelle**
- **SQL** est un langage qui n'explique pas comment obtenir les données
- Les opérateurs de l'algèbre relationnelle correspondent à ce qui est utilisé lorsqu'une requête est écrite en SQL : ils correspondent aux **opérations logiques** effectuées par le SGBD pour obtenir les données
- Les opérations sont faites sur une ou plusieurs relations, sans changer la relation initiale, pour obtenir une **nouvelle relation**

Les opérateurs

Les opérateurs de base

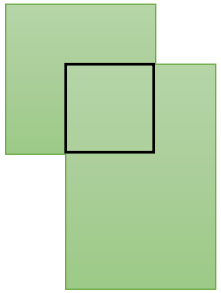
Restriction

Projection

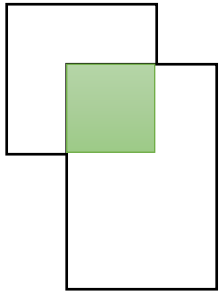


Les opérateurs ensemblistes

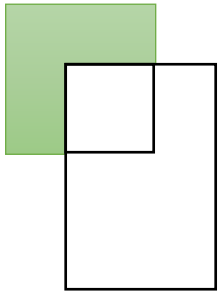
Union



Intersection



Différence



Les opérateurs de combinaison

Produit

A
B
C

X
Y

A	X
A	Y
B	X
B	Y
C	X
C	Y

Jointure

a	x
b	x
c	y

x	1
y	2
z	3

a	x	1
b	x	1
c	y	2

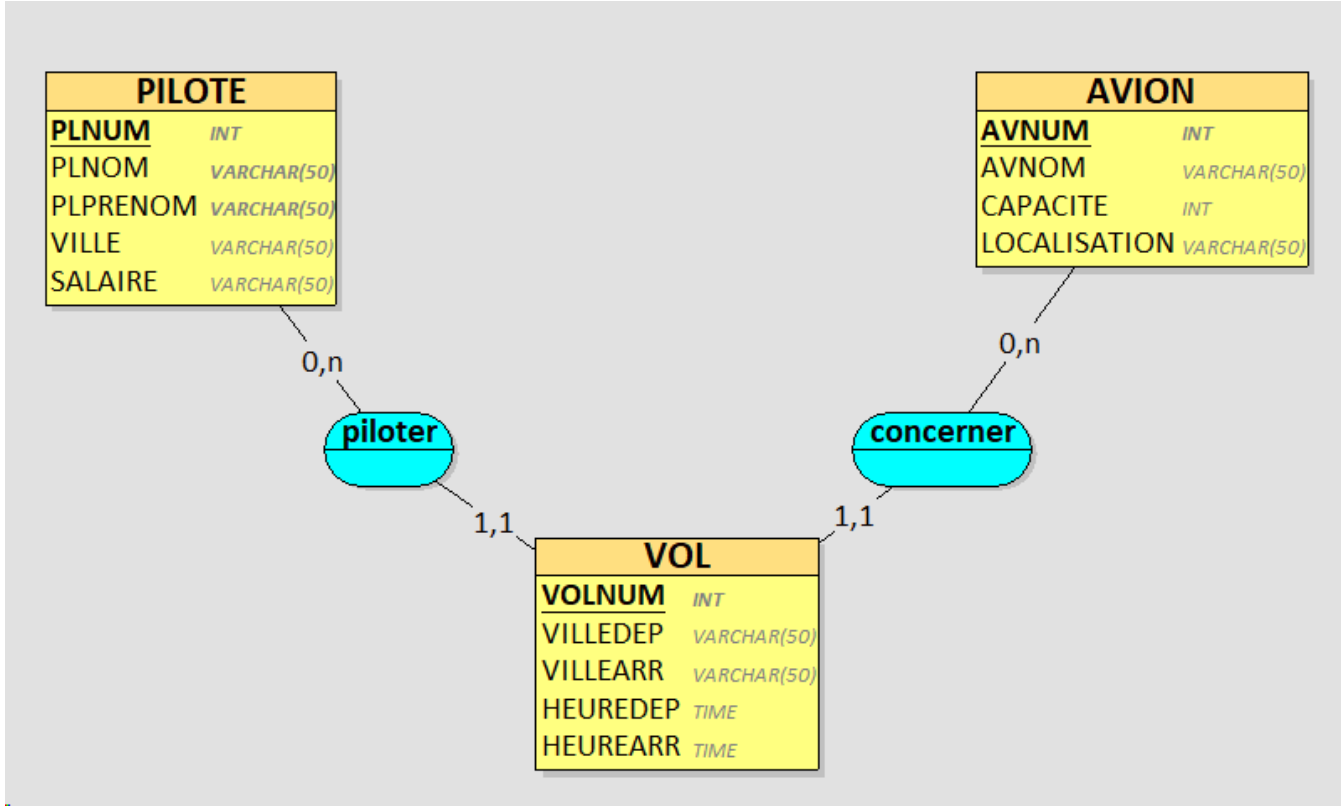
Les principaux opérateurs

- **Sélection ou restriction** (σ) : sélection d'un sous ensemble de tuples d'une relation. ➔ SQL : WHERE
- **Projection** (π) : suppression des attributs d'une relation ➔ SQL : SELECT distinct .. FROM …

- **Union** (∪) : sélection des tuples d'une relation R1 et de ceux d'une autre relation R2 ➔ SQL : UNION
- **Intersection** (∩) → SQL : INTERSECT
- **Différence** (− ou \) : sélection des tuples d'une relation R1 qui n'appartiennent pas à une relation R2 ➔ SQL : EXCEPT *(n'existe pas en Mysql)*

- **Jointure** (⋈) : combine deux relations ➔ SQL : JOIN
- **Produit cartésien** (×) ➔ SQL : select … FROM T1, T2 …

BD exemple : Gestion d'un aéroport MCD et MLD



pilote = (**PLNUM** *INT*, PLNOM *VARCHAR(50)*, PLPRENOM *VARCHAR(50)*, VILLE *VARCHAR(50)*, SALAIRE *CURRENCY*);

avion = (**AVNUM** *INT*, AVNOM *VARCHAR(50)*, CAPACITE *INT*, LOCALISATION *VARCHAR(50)*);

vol = (**VOLNUM** *INT*, VILLEDEP *VARCHAR(50)*, VILLEARR *VARCHAR(50)*, HEUREDEP *TIME*, HEUREARR *TIME*, *#PLNUM*, *#AVNUM*);

Les données de la base « aéroport »

pilote

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE
1	Dupont	Jean	Paris	5000
2	Martin	Paul	Marseille	4500
3	Dubois	Marie	Lyon	4800
4	Lefebvre	Sophie	Toulouse	5200
5	Moreau	Pierre	Nice	4900
6	Garcia	Julie	Bordeaux	5100
7	Fournier	Nicolas	Lille	4700
8	Leroy	Céline	Strasbourg	5300
9	Lopez	Thomas	Nantes	4600
10	Sanchez	Lucie	Rennes	5400

avion

AVNUM	AVNOM	CAPACITE	LOCALISATION
1	Airbus A320	150	Paris
2	Boeing 737	180	Marseille
3	Airbus A380	500	Lyon
4	Boeing 747	400	Toulouse
5	Embraer E190	100	Nice
6	Airbus A330	250	Bordeaux
7	Boeing 777	350	Lille
8	Airbus A350	300	Strasbourg
9	Boeing 767	280	Nantes
10	Embraer E175	90	Rennes

vol

VOLNUM	VILLEDEP	VILLEARR	HEUREDEP	HEUREARR	AVNUM	PLNUM
1	Paris	Marseille	08:00:00	10:00:00	1	1
2	Marseille	Paris	10:30:00	12:30:00	2	2
3	Lyon	Toulouse	09:15:00	10:45:00	3	3
4	Toulouse	Lyon	11:30:00	12:45:00	4	4
5	Nice	Bordeaux	10:00:00	12:00:00	5	5
6	Bordeaux	Nice	13:00:00	15:00:00	6	6
7	Lille	Strasbourg	11:45:00	13:30:00	7	7
8	Strasbourg	Lille	14:15:00	16:00:00	8	8
9	Nantes	Rennes	09:30:00	10:15:00	9	9
10	Rennes	Nantes	11:00:00	11:45:00	10	10

Partie 1 : Opérateurs de base

Projection

π

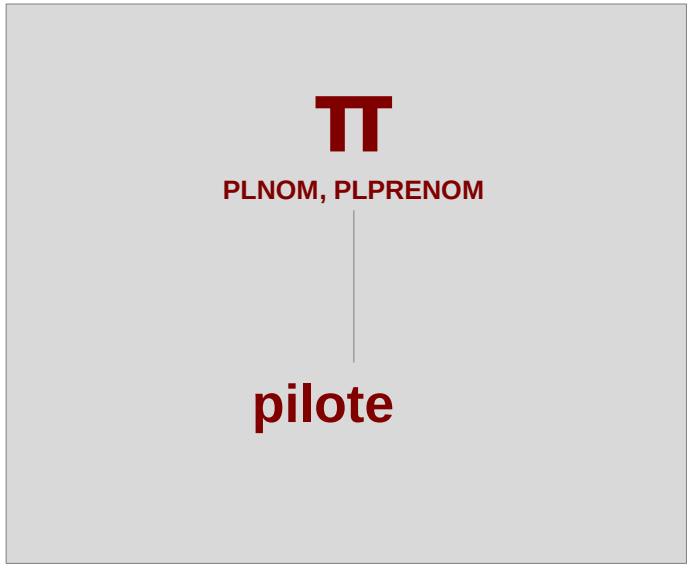
La projection permet de conserver
un sous-ensemble d'attributs d'une relation

Abus de langage
« la projection permet d'indiquer quelles *colonnes* utiliser »

PROJECTION

- C'est un opérateur **unaire** dont l'opérande est **une relation**,
- Nécessite la **liste des attributs** sur lesquels projeter en paramètre.

<u>Exemple</u> : « obtenir les noms et prénoms des pilotes. »



arbre relationnel

$$\pi_{\text{PLNOM, PLPRENOM}}(\text{pilote})$$

PLNOM	PLPRENOM
Dupont	Jean
Martin	Paul
Dubois	Marie
Lefebvre	Sophie
Moreau	Pierre
Garcia	Julie
Fournier	Nicolas
Leroy	Céline
Lopez	Thomas
Sanchez	Lucie

<u>En SQL :</u>
select PLNOM, PLPRENOM **from** PILOTE;

PROJECTION

Le résultat d'une projection est **une relation**,
c'est à dire **un ensemble de tuples** : il n'y a pas de doublons

---

π

PLNOM, PLPRENOM

pilote

PLNOM	PLPRENOM
Dupont	Jean
Martin	Paul
Dubois	Marie
Lefebvre	Sophie
Moreau	Pierre
Garcia	Julie
Fournier	Nicolas
Leroy	Céline
Lopez	Thomas
Sanchez	Lucie

π

PLPRENOM, PLNOM

pilote

PLPRENOM	PLNOM
Jean	Dupont
Paul	Martin
Marie	Dubois
Sophie	Lefebvre
Pierre	Moreau
Julie	Garcia
Nicolas	Fournier
Céline	Leroy
Thomas	Lopez
Lucie	Sanchez

l'ordre des attributs projetés est important

Projection simple : SELECT

```
-- projection simple avec choix des colonnes
select AVNUM, AVNOM, LOCALISATION from avion;
```

AVNUM	AVNOM	LOCALISATION
1	A300	NICE
2	A310	NICE
3	B707	PARIS
4	A300	LYON
5	CONCORDE	NICE
6	B747	PARIS
7	B707	PARIS
8	A310	TOULOUSE
9	MERCURE	LYON
10	CONCORDE	PARIS

On peut préciser explicitement le nom de la table à partir de laquelle est issu l'attribut. Il faut alors préfixer le nom de l'attribut par le nom de la table.

```
select avion.AVNUM, avion.AVNOM, avion.LOCALISATION from avion;
```

```
select * from avion;
```

*** = liste des colonnes**

Projection simple : SELECT

Il est possible de renommer (alias) un attribut en utilisant le mot-clé **AS.**
Ce mot clé est facultatif entre l'attribut et son alias.

```
select
    PLNOM,
    PLPRENOM
from
    pilote ;
```

PLNOM	PLPRENOM
MIRANDA	SERGE
LETHANH	NAHN
TALADOOIRE	GILLES
BONFILS	ELIANE
LAKHAL	LOTFI
BONFILS	GERARD

```
select
    PLNOM as Nom,
    PLPRENOM as Prenom
from
    pilote ;
```

Nom	Prenom
MIRANDA	SERGE
LETHANH	NAHN
TALADOOIRE	GILLES
BONFILS	ELIANE
LAKHAL	LOTFI
BONFILS	GERARD
MORCENAC	PIERRE

Restriction/selection

σ

La restriction permet de conserver
un sous-ensemble de tuples vérifiant un prédicat

Abus de langage
« la restriction permet d'indiquer quelles *lignes* utiliser »

RESTRICTION

- C'est un opérateur **unaire** dont l'opérande est **une relation**

- Nécessite un **prédicat**, sous la forme ***x op y***, que les tuples doivent vérifier :
  - x un attribut ou bien une valeur
  - y un attribut ou bien une valeur
  - op un opérateur parmi =, ≠ , <, >, ≤ , ≥

Exemple : "Obtenir les informations des pilotes qui gagnent au moins 5000 euros."

$\sigma_{\text{SALAIRE} \geq 5000}$ (pilote)

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE
1	Dupont	Jean	Paris	5000
4	Lefebvre	Sophie	Toulouse	5200
6	Garcia	Julie	Bordeaux	5100
8	Leroy	Céline	Strasbourg	5300
10	Sanchez	Lucie	Rennes	5400

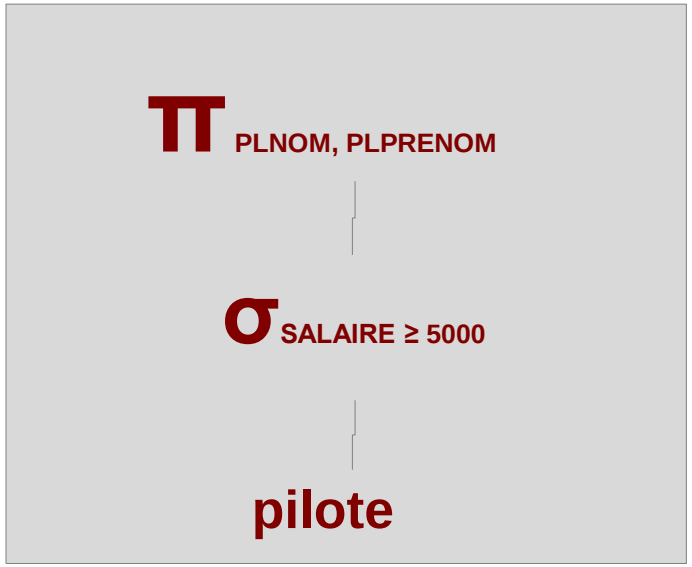
R1

<u>En SQL :</u> SELECT * FROM pilote **WHERE** SALAIRE >=5000;

RESTRICTION

On peut choisir les informations à projeter !

Exemple : "Obtenir les noms et prénoms des pilotes qui gagnent au moins 5000 euros."



arbre relationnel

$$\pi_{\text{PLNOM, PLPRENOM}}(\sigma_{\text{SALAIRE} \geq 5000}(\text{pilote}))$$

PLNOM	PLPRENOM
Dupont	Jean
Lefebvre	Sophie
Garcia	Julie
Leroy	Céline
Sanchez	Lucie

<u>En SQL :</u> **SELECT** PLNOM, PLPRENOM **FROM** pilote **WHERE** SALAIRE **>=** 5000;

Projection simple conditionnée

```
-- Projection simple conditionnée
select AVNOM
from avion
where LOCALISATION = 'Paris';
```

AVNOM
B707
B747
B707
CONCORDE

Il y a une différence de comportement entre SQL et l'algèbre relationnelle : ***Pour SQL, les doublons existent et sont affichés***.

```
-- Projection simple conditionnée et sans doublons
select distinct AVNOM
from avion
where LOCALISATION = 'Paris';
```

AVNOM
B707
B747
CONCORDE

Projection avec conditions plus complexes : null

```
-- projection avec conditions plus complexes : null
select PLNOM
from pilote
where VILLE = null;
```

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIR
1	Dupont	Jean	Paris	5000
2	Martin	Paul	Marseille	4500
3	Dubois	Marie	Lyon	4800
4	Lefebvre	Sophie	Toulouse	5200
5	Moreau	Pierre	Nice	4900
6	Garcia	Julie	Bordeaux	5100
7	Fournier	Nicolas	Lille	4700
8	Leroy	Céline	Strasbourg	5300
9	Lopez	Thomas	Nantes	4600
10	Sanchez	Lucie	Rennes	5400
11	Dupont	Jean	NULL	5000
12	Martin	Paul	NULL	4500
13	Dubois	Marie	NULL	4800
14	Lefebvre	Sophie	NULL	5200
15	Moreau	Pierre	NULL	4900
16	Garcia	Julie	NULL	5100
17	Fournier	Nicolas	NULL	4700
18	Leroy	Céline	NULL	5300
19	Lopez	Thomas	NULL	4600
20	Sanchez	Lucie	NULL	5400

null **ne peut être comparé à rien, même pas à lui-même !**

```
select PLNOM
from pilote
where VILLE is null;

select PLNOM
from pilote
where VILLE is not null;
```

PLNOM
Dupont
Martin
Dubois
Lefebvre
Moreau
Garcia
Fournier
Leroy
Lopez
Sanchez

Projection avec conditions plus complexes - in et between

```
select PLNOM Nom, PLPRENOM Prénom
From Pilote
Where Ville in ('PAris', 'Nantes', 'Toulouse');
```

Nom	Prénom
Dupont	Jean
Lefebvre	Sophie
Lopez	Thomas

```
select PLNOM Nom, PLPRENOM Prénom
From Pilote
Where Ville not in ('PAris', 'Nantes', 'Toulouse');
```

Nom	Prénom
Martin	Paul
Dubois	Marie
Moreau	Pierre
Garcia	Julie
Fournier	Nicolas
Leroy	Céline
Sanchez	Lucie

```
select PLNOM Nom, PLPRENOM Prénom
From Pilote
Where SALAIRE between 5000 and 6000
```

Nom	Prénom
Dupont	Jean
Lefebvre	Sophie
Garcia	Julie
Leroy	Céline
Sanchez	Lucie
Dupont	Jean
Lefebvre	Sophie
Garcia	Julie
Leroy	Céline
Sanchez	Lucie

Projection avec conditions plus complexes – les masques

```
select PLPRENOM
from   pilote
where   PLNOM like 'B_';
```

'_' = un caractère quelconque

masques

```
select PLPRENOM
from   pilote
where  PLNOM like '%AU%';
```

'%' = une chaîne quelconque

Un masque définit une famille de chaînes de caractères :

'B_'	→	'B1' 'Bd' 'B '
'B_'	↛	'xB' 'B'

'%AU%'	→	'SEAU' 'Plein AU' 'AU LENT'
'%AU%'	↛	'BE A UPE A U' 'Achetez S A P A U !'

Projection avec conditions plus complexes – Combinaisons logiques

Informations sur les pilotes habitant ‘Toulouse’ et ayant un salaire > 5000

```
select PLNOM, PLPRENOM, VILLE, SALAIRE
from   pilote
where  VILLE = 'Toulouse' and SALAIRE > 5000;
```

PLNOM	PLPRENOM	VILLE	SALAIRE
Lefebvre	Sophie	Toulouse	5200

Informations sur les pilotes ayant un salaire >= 5000 et qui habitent soit ‘Toulouse’ soit ‘Paris’

```
select PLNOM, PLPRENOM, VILLE, SALAIRE
from   pilote
where  SALAIRE >= 5000
    and    (VILLE = 'TOULOUSE' or VILLE = 'Paris');
```

PLNOM	PLPRENOM	VILLE	SALAIRE
Dupont	Jean	Paris	5000
Lefebvre	Sophie	Toulouse	5200

Données extraites et données dérivées

```
select PLNOM, 'Salaire Net', '=', SALAIRE - 0.25 * SALAIRE
from pilote
where SALAIRE > 5000;
```

PLNOM	Salaire Net	=	SALAIRE - 0.25 * SALAIRE
Lefebvre	Salaire Net	=	3900
Garcia	Salaire Net	=	3825
Leroy	Salaire Net	=	3975
Sanchez	Salaire Net	=	4050
Lefebvre	Salaire Net	=	3900

Mieux encore, en utilisant des alias !

```
select PLNOM as 'Nom Pilote', SALAIRE - 0.25 * SALAIRE as 'Salaire Net'
from pilote
where SALAIRE > 5000;
```

Nom Pilote	Salaire Net
Lefebvre	3900
Garcia	3825
Leroy	3975
Sanchez	4050
Lefebvre	3900

Données extraites et données dérivées – les fonctions

Outre les 4 opérations arithmétiques, SQL offre une large gamme de fonctions

Fonctions				
Maths	Dates et heures		Chaînes de caractères	
ABS	ADDDATE	MONTH	ASCII	MATCH
ACOS	ADDTIME	MONTHNAME	BIN	MID
ASIN	CONVERT_TZ	NOW	BIT_LENGTH	OCT
ATAN	CURDATE	PERIOD_ADD	CHAR_LENGTH	OCTET_LENGTH
CEIL	CURRENT_DATE	PERIOD_DIFF	CHAR	ORD
CEILING	CURTIME	QUARTER	CHARACTER_LENGTH	POSITION
CONV	CURRENT_TIME	SEC_TO_TIME	CONCAT_WS	QUOTE
COS	CURRENT_TIMESTAMP	SECOND	CONCAT	REPEAT
COT	DATE_ADD	STR_TO_DATE	ELT	REPLACE
CRC32	DATE_FORMAT	SUBDATE	EXPORT_SET	REVERSE
DEGREES	DATE_SUB	SUBTIME	FIELD	RIGHT
EXP	DATE	SYSDATE	FIND_IN_SET	RPAD
FLOOR	DATEDIFF	TIME_FORMAT	FORMAT	RTRIM
LN	DAYNAME	TIME_TO_SEC	HEX	SOUNDEX
LOG10	DAYOFMONTH, DAY	TIME	INSERT	SPACE
LOG2	DAYOFWEEK	TIMEDIFF	INSTR	STRCMP
LOG	DAYOFYEAR	TIMESTAMP	LCASE	SUBSTR
MOD	EXTRACT	TIMESTAMPADD	LEFT	SUBSTRING_INDEX
PI	FROM_DAYS	TIMESTAMPDIFF	LENGTH	SUBSTRING
POW	FROM_UNIXTIME	TO_DAYS	LIKE	TRIM
POWER	GET_FORMAT	UNIX_TIMESTAMP	LOAD_FILE	UCASE
RADIANS	HOUR	UTC_DATE	LOCATE	UNHEX
RAND	LAST_DAY	UTC_TIME	LOWER	UPPER
SIGN	LOCALTIME	UTC_TIMESTAMP	LPAD	
SIN	LOCALTIMESTAMP	WEEK	LTRIM	
SQRT	MAKEDATE	WEEKDAY	MAKE_SET	
TAN	MAKETIME	WEEKOFYEAR		
TRUNCATE	MICROSECOND	YEAR		
	MINUTE	YEARWEEK		

https://sql.sh/919-aide-memoire-mysql

Données extraites et données dérivées – les fonctions

Quelques exemples …

```
SELECT PLNOM as 'Nom pilote'
from pilote
where LENGTH(PLNOM) = 7;
```

Nom pilote
Sanchez

```
select UPPER(PLNOM) as Nom, PLPRENOM
from Pilote;
```

Nom	PLPRENOM
DUPONT	Jean
MARTIN	Paul
DUBOIS	Marie
LEFEBVRE	Sophie
MOREAU	Pierre
GARCIA	Julie

```
SELECT DATE_FORMAT("2018-09-24", "%D %b %Y");
```

Conversion
24th Sep 2018

```
-- format de la réponse : "HH:MM:SS"
SELECT  VILLEDEP as 'Départ',
        VILLEARR as 'Destination',
         TIMEDIFF(HEUREARR, HEUREDEP) as 'Durée Vol'
from vol;
```

Départ	Destination	Durée Vol
Paris	Marseille	02:00:00
Marseille	Paris	02:00:00
Lyon	Toulouse	01:30:00
Toulouse	Lyon	01:15:00
Nice	Bordeaux	02:00:00
Bordeaux	Nice	02:00:00
Lille	Strasbourg	01:45:00

Partie 2 : Opérateurs de l'algèbre relationnelle étendue

Les opérateurs

γ, τ

Les fonctions d'agrégation (statistiques)

- AVG() pour calculer la moyenne sur un ensemble d'enregistrement
- COUNT() pour compter le nombre d'enregistrement sur une table ou une colonne distincte
- MAX() pour récupérer la valeur maximum d'une colonne sur un ensemble de ligne. Cela s'applique à la fois pour des données numériques ou alphanumérique
- MIN() pour récupérer la valeur minimum de la même manière que MAX()
- SUM() pour calculer la somme sur un ensemble d'enregistrement

```
select 'Paris',
       avg(CAPACITE) as CAPACITE_Moyenne,
       max(CAPACITE)-min(CAPACITE) as Ecart_max,
       count(*) as Nombre
from   avion
where  LOCALISATION = 'Paris';
```

Paris	CAPACITE_Moyenne	Ecart_max	Nombre
Paris	150.0000	0	1

le résultat ne comprend qu'une seule ligne

Les fonctions d'agrégation (statistiques)

Attention aux valeurs dupliquées

```
select count(PLNUM)
from   pilote;
```

=

```
select distinct count(PLNUM)
from   vol;
```

count(PLNUM)
16

```
select count(distinct PLNUM)
from   vol;
```

count(distinct PLNUM)
9

Avec doublons : distinct ne prend pas effet

Sans doublons : distinct prend effet

Les fonctions d'agrégation (statistiques)

```
select count(VOLNUM) as Vol,
       count(VILLEDEP) as Départ,
       count(VILLEARR) as Arrivée,
       count(AVNUM) as Avion
from   vol;
```

Vol	Départ	Arrivée	Avion
16	16	16	16

```
select count(distinct VOLNUM) as Vol,
       count(distinct  VILLEDEP) as Départ,
       count(distinct  VILLEARR) as Arrivée,
       count(distinct  AVNUM) as Avion
from   vol;
```

Vol	Départ	Arrivée	Avion
16	6	6	7

Les fonctions d'agrégation (statistiques)

Cas des ensembles vides

```
select count(*) as Nombre,
        sum(SALAIRE) as SALAIRE,
        max(SALAIRE) as Max
from   pilote
where  VILLE = 'Paris';
```

Nombre	SALAIRE	Max
4	76000	21000

```
select count(*) as Nombre,
        sum(SALAIRE) as SALAIRE,
        max(SALAIRE) as Max
from   pilote
where  VILLE = 'Tunis';
```

Nombre	SALAIRE	Max
0	NULL	NULL

Agrégation et regroupement en algèbre relationnelle

γ = agrégation (AVG, COUNT, MIN, ….) et regroupement

R = (

A	B	C
1	2	3
4	5	6
1	2	5

)

$\gamma_{A,B,AVG(C)\text{->}X}$ (R) = ??

First, group *R* by *A* and *B* :

A	B	C
1	2	3
1	2	5
4	5	6

Then, average *C* within groups:

A	B	X
1	2	4
4	5	6

Les données groupées - Principe

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE
6	Garcia	Julie	Bordeaux	5100
7	Fournier	Nicolas	Lille	4700
3	Dubois	Marie	Lyon	4800
14	Girard	Emma	Lyon	4800
2	Martin	Paul	Marseille	4500
15	Roy	Lucas	Marseille	5200
9	Lopez	Thomas	Nantes	4600
5	Moreau	Pierre	Nice	4900
1	Dupont	Jean	Paris	5000
16	Caron	Manon	Paris	5000
10	Sanchez	Lucie	Rennes	5400
8	Leroy	Céline	Strasbourg	5300
4	Lefebvre	Sophie	Toulouse	5200

le groupe des pilotes de LYON

le groupe des pilotes de MARSEILLE

le groupe des Pilotes de PARIS

SQL : Les données groupées - Principe

La clause **GROUP BY** est utilisée pour **regrouper** un ensemble de lignes de résultats en fonction des valeurs de une ou plusieurs colonnes. Cela permet **d'effectuer des opérations d'agrégation sur chaque groupe de lignes**, telles que la somme, le comptage, la moyenne, etc.

1. **Regroupement des lignes** : Les lignes sont regroupées en fonction des valeurs uniques des colonnes spécifiées dans la clause GROUP BY.
2. **Agrégation des données** : Une fois les lignes regroupées, des fonctions d'agrégation telles que SUM, COUNT, AVG, MAX, MIN, etc., peuvent être appliquées à chaque groupe pour calculer des valeurs agrégées basées sur les valeurs des colonnes de ce groupe.
3. **Résultats par groupe** : Le résultat final de la requête affiche une ligne pour chaque groupe, avec les valeurs agrégées calculées pour ce groupe. Il ne peut y avoir dans le SELECT que des attributs du GROUP BY ou des fonctions d’agrégation.

SQL : GROUP BY – Exemples

« Nombre de pilotes par ville »

```sql
select VILLE, COUNT(*) as 'Nombre Pilotes'
from pilote
group by VILLE;
```

VILLE	Nombre Pilotes
Paris	2
Marseille	2
Lyon	2
Toulouse	1
Nice	1
Bordeaux	1
Lille	1
Strasbourg	1
Nantes	1
Rennes	1

« Moyenne des salaires des pilotes par ville »

```sql
select VILLE, AVG(SALAIRE) as 'Salaire Moyen des pilotes'
from pilote
group by VILLE;
```

VILLE	Salaire Moyen des pilotes
Paris	5000
Marseille	4850
Lyon	4800
Toulouse	5200
Nice	4900
Bordeaux	5100
Lille	4700
Strasbourg	5300
Nantes	4600
Rennes	5400

SQL : Sélection des groupes avec filtrage

La clause **HAVING** est utilisée en SQL pour **filtrer les résultats d'une requête GROUP BY** en fonction de conditions spécifiées. Contrairement à la clause WHERE, qui filtre les lignes avant le regroupement, la clause HAVING filtre les groupes de lignes après le regroupement.

1. **Filtrage après regroupement** : La clause HAVING est utilisée pour appliquer des conditions de filtrage sur les groupes de résultats générés par la clause GROUP BY.
2. **Utilisation avec des fonctions d'agrégation** : La clause HAVING est souvent utilisée en conjonction avec des fonctions d'agrégation telles que SUM, COUNT, AVG, MAX, MIN, etc., pour filtrer les groupes de résultats en fonction de valeurs agrégées.
3. **Syntaxe similaire à WHERE** : La syntaxe de la clause HAVING est similaire à celle de la clause WHERE, mais elle est utilisée ***après la clause GROUP BY***.

SQL : GROUP BY et HAVING + fonction d'agrégation

« Moyenne des salaires des pilotes pour les villes ayant 1 ou plus pilotes. »

```
select VILLE, count(*) as 'Nombre pilotes', avg(SALAIRE) as 'Salaire Moyen des pilotes'
from   pilote
group by VILLE
having count(*) > 1;
```

VILLE	Nombre pilotes	Salaire Moyen des pilotes
Paris	2	5000
Marseille	2	4850
Lyon	2	4800

Tri des résultats - Principe

Algèbre relationnelle :

τ = tri les tuples (en SQL : ORDER BY(asc, desc))

SQL : La clause **ORDER BY** est utilisée pour **trier les résultats** d'une requête en fonction des valeurs d'une ou plusieurs colonnes. Elle peut être utilisée avec SELECT pour trier les résultats d'une requête, avec ou sans GROUP BY.

1. **Tri des résultats** : La clause ORDER BY est utilisée pour trier les lignes de résultats selon les valeurs de une ou plusieurs colonnes, dans un ordre ascendant ou descendant.
2. **Tri ascendant ou descendant** : Vous pouvez spécifier l'ordre de tri à l'aide des mots-clés ASC (par défaut) ou DESC.
3. **Tri basé sur des colonnes** : Vous pouvez spécifier ***une ou plusieurs colonnes*** pour trier les résultats. Si plusieurs colonnes sont spécifiées, le tri se fait par ordre de priorité, en commençant par la première colonne spécifiée.

Tri les résultats - Exemples

```
select * FROM PILOTE order by VILLE ;
```

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE
6	Garcia	Julie	Bordeaux	5100
7	Fournier	Nicolas	Lille	4700
3	Dubois	Marie	Lyon	4800
14	Girard	Emma	Lyon	4800
2	Martin	Paul	Marseille	4500
15	Roy	Lucas	Marseille	5200
9	Lopez	Thomas	Nantes	4600
5	Moreau	Pierre	Nice	4900
1	Dupont	Jean	Paris	5000
16	Caron	Manon	Paris	5000
10	Sanchez	Lucie	Rennes	5400
8	Leroy	Céline	Strasbourg	5300
4	Lefebvre	Sophie	Toulouse	5200

```
select * FROM PILOTE order by VILLE desc;
```

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE
4	Lefebvre	Sophie	Toulouse	5200
8	Leroy	Céline	Strasbourg	5300
10	Sanchez	Lucie	Rennes	5400
1	Dupont	Jean	Paris	5000
16	Caron	Manon	Paris	5000
5	Moreau	Pierre	Nice	4900
9	Lopez	Thomas	Nantes	4600
2	Martin	Paul	Marseille	4500
15	Roy	Lucas	Marseille	5200
3	Dubois	Marie	Lyon	4800
14	Girard	Emma	Lyon	4800
7	Fournier	Nicolas	Lille	4700
6	Garcia	Julie	Bordeaux	5100

Partie 3 : Opérateurs de combinaison

Produit cartésien

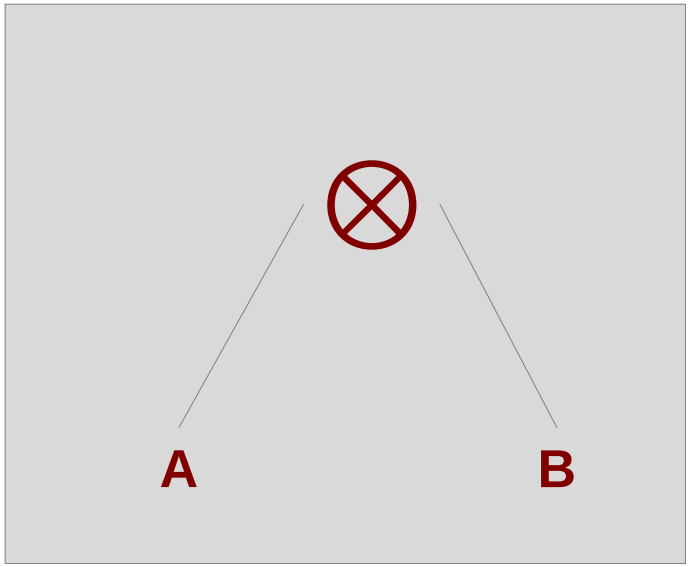
$\otimes$

Le produit cartésien de deux relations permet d'obtenir une nouvelle relation dont le schéma est la **concaténation des schémas** des relations opérandes, et dont les tuples sont **tous les tuples que l'on peut construire** en combinant un tuple de la première relation avec de la deuxième relation.

Abus de langage :
« le produit cartésien permet de construire une relation
$A \otimes B$
ayant toutes les colonnes de A et B, et toute combinaison de leurs lignes »

Produit cartésien

C'est un opérateur **binaire** dont les opérandes sont **deux relations**



arbre relationnel

A	att 1	att 2
	a	1
	b	2
	b	3

B	att 3	att 4
	x	5
	y	6

A⊗B	att1	att2	att3	att4
	a	1	x	5
	a	1	y	6
	b	2	x	5
	b	2	y	6
	b	3	x	5
	b	3	y	6

Produit cartésien

avion ⊗ pilote

Select * from AVION, PILOTE

Tuples construits avec tous les tuples de « avion » et le tuple 1 de pilote

AVNUM	AVNOM	CAPACITE	LOCALISATION	PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE
10	Embraer E175	90	Rennes	1	Dupont	Jean	Paris	5000
9	Boeing 767	280	Nantes	1	Dupont	Jean	Paris	5000
8	Airbus A350	300	Strasbourg	1	Dupont	Jean	Paris	5000
7	Boeing 777	350	Lille	1	Dupont	Jean	Paris	5000
6	Airbus A330	250	Bordeaux	1	Dupont	Jean	Paris	5000
5	Embraer E190	100	Nice	1	Dupont	Jean	Paris	5000
4	Boeing 747	400	Toulouse	1	Dupont	Jean	Paris	5000
3	Airbus A380	500	Lyon	1	Dupont	Jean	Paris	5000
2	Boeing 737	180	Marseille	1	Dupont	Jean	Paris	5000
1	Airbus A320	150	Paris	1	Dupont	Jean	Paris	5000
10	Embraer E175	90	Rennes	2	Martin	Paul	Mars...	4500
9	Boeing 767	280	Nantes	2	Martin	Paul	Mars...	4500
8	Airbus A350	300	Strasbourg	2	Martin	Paul	Mars...	4500
7	Boeing 777	350	Lille	2	Martin	Paul	Mars...	4500

Exemple d'utilisation du produit cartésien :

$$\pi_{PLNOM}(\sigma_{VILLE = « Paris » \text{ and } AVNUM = 10}(avion \otimes pilote))$$

SQL : Select PLNOM from AVION, PILOTE where VILLE = « Paris » and AVNUM = 10;

JOINTURE

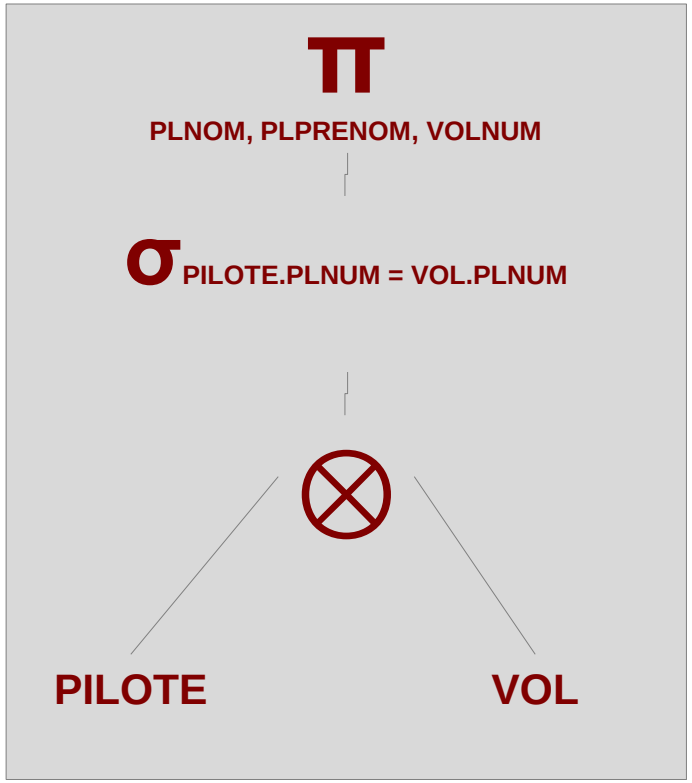
$\bowtie$

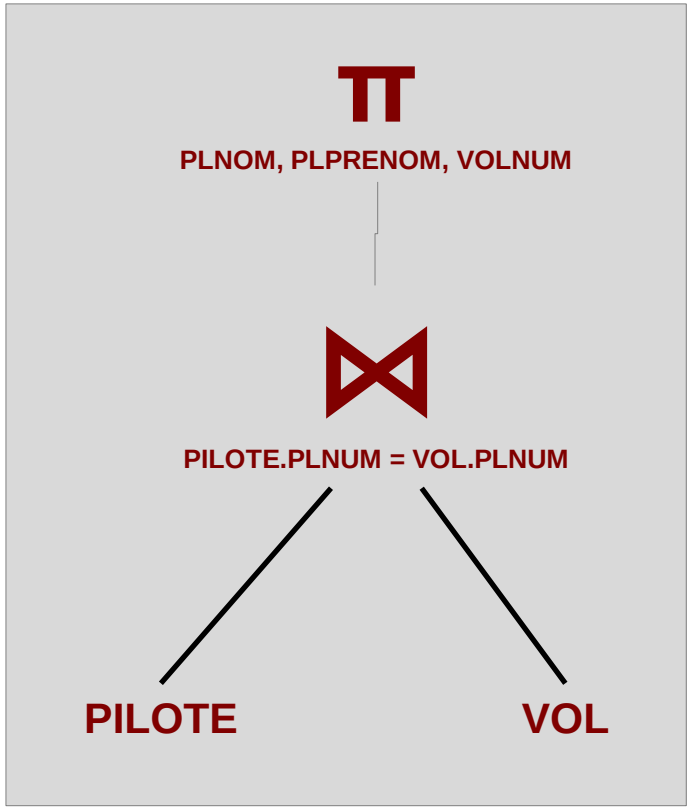
Un opérateur spécifique pour associer des relations selon un lien logique

JOINTURE interne

- C'est un opérateur **binaire** dont les opérandes sont **deux relations**,
- Sert à ramener dans la même relation des informations issues d'autres relations.
- C'est un raccourci d'écriture.

<u>Exemple</u> : « Afficher le nom et le prénom des pilotes ainsi que les numéros des vols qu'ils ont effectués. »





JOINTURE interne

<u>Exemple</u> : « Afficher le nom et le prénom des pilotes ainsi que les numéros des vols qu'ils ont effectués. »

$$\pi_{\text{PLNOM, PLPRENOM, VOLNUM}}(\sigma_{\text{PILOTE.PLNUM = VOL.PLNUM}}(\text{vol} \otimes \text{pilote}))$$

$$\Downarrow$$

$$\pi_{\text{PLNOM, PLPRENOM, VOLNUM}}(\text{vol} \bowtie_{\text{PILOTE.PLNUM = VOL.PLNUM}} \text{pilote})$$

JOINTURE interne

<u>Exemple :</u> « Afficher le nom et le prénom des pilotes ainsi que les numéros des vols qu'ils ont effectués. »

1

```
Select PLNOM, PLPRENOM, VOLNUM
from PILOTE, VOL
where PILOTE.PLNUM = VOL.PLNUM;
```

plusieurs tables

condition de jointure

préfixe nécessaire car ambiguïté

2

```
Select PLNOM, PLPRENOM, VOLNUM
from PILOTE P JOIN VOL V
    ON P.PLNUM = V.PLNUM;
```

plusieurs tables

condition de jointure

Utilisation des alias nécessaire car ambiguïté

3

```
Select PLNOM, PLPRENOM, VOLNUM
from PILOTE JOIN VOL
    USING(PLNUM) ;
```

EQUI-JOINTURE SUR ATTRIBUTS DE MEME NOM

***JOIN* par défaut correspond à *INNER JOIN* (jointure interne)**

Différents types de jointures

Il y a plusieurs types de jointures, notamment en fonction de l’expression utilisée pour la condition de jointure. Nous traiterons les suivantes :

1- La **jointure interne** ou la **jointure naturelle**, pour garder les lignes qui correspondent entre plusieurs tables

2- L’**auto-jointure** : est une jointure d'une table avec elle-même

3- La **jointure externe** : par rapport à la jointure interne, on complète les attributs vides par des NULL : elle peut être à gauche (⟕), à droite (⟖) ou complète (⟗).

A LEFT OUTER JOIN B =➔ remplir la partie gauche avec NULL

A RIGHT OUTER JOIN B ➔ remplir la partie droite avec NULL

Jointures interne et naturelles

Jointures **interne** et **naturelle** servent à relier deux relations en gardant des lignes qui correspondent.

Dans la **jointure interne**, on spécifie la condition de jointure, alors qu'avec une **jointure naturelle**, elle se fait automatiquement sur les attributs qui ont le même nom dans les deux relations :

Exemple en SQL :

```
select PLNOM, PLPRENOM, VOLNUM
from PILOT NATURAL JOIN VOL;
```

- S'il n'y a que PLNUM qui est commun aux deux tables, la jointure se fait sur PLNUM
- ***Attention: Si d'autres attributs ont le même nom*** *(par exemple un nom de personne)****, ils seront automatiquement utilisés dans la jointure ! L'opération faite sur 2 relations A et B correspond à :*** ON A.col1 = B.col1 AND A.col2 = B.col2…

En algèbre relationnelle :

$$\pi_{\text{PLNOM, PLPRENOM, VOLNUM}}(\text{vol} \bowtie_{\text{PILOTE.PLNUM = VOL.PLNUM}} \text{pilote})$$

INNER JOIN – recommandé !

$$\pi_{\text{PLNOM, PLPRENOM, VOLNUM}}(\text{vol} \bowtie \text{pilote})$$

NATURAL JOIN

Auto-jointures

L'auto-jointure est une jointure d'une relation avec elle-même. Concrètement, cela se passe comme si on avait deux copies d'une même table.

Exemple en SQL : on veut les noms des pilotes qui habitent dans la même ville, listés par paire

```sql
SELECT
    p1.PLNOM AS nom1,
    p2.PLNOM AS nom2,
    p1.VILLE
FROM pilote p1
JOIN pilote p2
ON p1.VILLE = p2.VILLE
AND p1.PLNUM <> p2.PLNUM;
```

ρ : renomme un attribut ou une relation

En algèbre relationnelle :

$$\rho_{P1}(\text{pilote}) \bowtie_{P1.VILLE = P2.VILLE \wedge P1.PLNUM <> P2.PLNUM} \rho_{P2}(\text{pilote})$$

C'est donc une **INNER JOIN**

Jointures externes

Jointure externe à gauche (left outer join)

Permet d'obtenir au moins une fois tous les tuples de la relation à gauche de la jointure. Sans correspondant dans la relation de droite, des valeurs nulles sont insérées.

« Afficher les noms et prénoms de tous les pilotes, avec le nom des villes de départ des vols qu'ils assurent, le cas échéant »

```
SELECT
    PLNOM,
    PLPRENOM,
    VILLEDEP
FROM
    pilote LEFT OUTER JOIN vol USING(PLNUM);
```

Colonnes de la table pilote

Colonnes de la table vol

PLNOM	PLPRENOM	VILLEDEP
Dupont	Jean	Paris
Martin	Paul	Marseille
Dubois	Marie	Lyon
Lefebvre	Sophie	Toulouse
Moreau	Pierre	Nice
Garcia	Julie	Bordeaux
Fournier	Nicolas	Lille
Leroy	Céline	Strasbourg
Lopez	Thomas	Nantes
Sanchez	Lucie	Rennes
Dupont	Jean	NULL
Martin	Paul	NULL
Dubois	Marie	NULL
Girard	Emma	NULL
Roy	Lucas	NULL
Caron	Manon	NULL

Jointures externes

Jointure externe à droite
(right outer join)

Permet d'obtenir au moins une fois tous les tuples de la relation à droite de la jointure. Sans correspondant dans la relation de gauche, des valeurs nulles sont insérées

Jointure externe complète
(full outer join)

Permet d'obtenir au moins une fois tous les tuples de la relation à droite de la jointure et au moins une fois tous les tuples de la relation à gauche de la jointure.

Les jointures - récapitulatif

A	att1	att2
	1	2
	3	3
	5	3
	10	4

B	att3	att4
	2	2
	3	25
	5	20

```
SELECT *
FROM A JOIN B
ON A.att2 =
B.att3 ;
```

Résultat	att1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25

```
SELECT *
FROM A FULL OUTER JOIN B
ON A.att2 = B.att3 ;
```

Résultat	att1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25
	10	4	null	null
	null	null	5	20

```
SELECT *
FROM A RIGHT JOIN B
ON A.att2 = B.att3 ;
```

Résultat	att1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25
	Null	Null	5	20

```
SELECT *
FROM A LEFT JOIN B
ON A.att2 = B.att3 ;
```

Résultat	att1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25
	10	4	Null	null

JOINTURE de plus de 2 tables

```
select *
from   pilote, avion, vol
where  pilote.PLNUM = vol.PLNUM
and    vol.AVNUM = avion.AVNUM;
```

PLNUM

AVNUM

PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE	AVNUM	AVNOM	CAPACITE	LOCALISATION	VOLNUM	VILLEDEP	VILLEARR	HEUREDEP	HEUREARR	AVNUM	PLNUM
1	Dupont	Jean	Paris	5000	1	Airbus A320	150	Paris	1	Paris	Marseille	08:00:00	10:00:00	1	1
1	Dupont	Jean	Paris	5000	1	Airbus A320	150	Paris	11	Paris	Marseille	08:00:00	10:00:00	1	1
2	Martin	Paul	Marseille	4500	2	Boeing 737	180	Marseille	2	Marseille	Paris	10:30:00	12:30:00	2	2
2	Martin	Paul	Marseille	4500	2	Boeing 737	180	Marseille	12	Marseille	Paris	10:30:00	12:30:00	2	2
3	Dubois	Marie	Lyon	4800	3	Airbus A380	500	Lyon	3	Lyon	Toulouse	09:15:00	10:45:00	3	3
3	Dubois	Marie	Lyon	4800	3	Airbus A380	500	Lyon	13	Lyon	Toulouse	09:15:00	10:45:00	3	3
4	Lefebvre	Sophie	Toulouse	5200	4	Boeing 747	400	Toulouse	4	Toulouse	Lyon	11:30:00	12:45:00	4	4
4	Lefebvre	Sophie	Toulouse	5200	4	Boeing 747	400	Toulouse	14	Toulouse	Lyon	11:30:00	12:45:00	4	4
5	Moreau	Pierre	Nice	4900	5	Embraer E190	100	Nice	5	Nice	Bordeaux	10:00:00	12:00:00	5	5
5	Moreau	Pierre	Nice	4900	5	Embraer E190	100	Nice	15	Nice	Bordeaux	10:00:00	12:00:00	5	5
6	Garcia	Julie	Bordeaux	5100	6	Airbus A330	250	Bordeaux	6	Bordeaux	Nice	13:00:00	15:00:00	6	6

JOINTURE de plus de 2 tables

```
select *
from pilote
join vol using(PLNUM)
join avion using(AVNUM);
```

PLNUM

AVNUM	PLNUM	PLNOM	PLPRENOM	VILLE	SALAIRE	VOLNUM	VILLEDEP	VILLEARR	HEUREDEP	HEUREARR	AVNOM	CAPACITE	LOCALISATION
1	1	Dupont	Jean	Paris	5000	1	Paris	Marseille	08:00:00	10:00:00	Airbus A320	150	Paris
1	1	Dupont	Jean	Paris	5000	11	Paris	Marseille	08:00:00	10:00:00	Airbus A320	150	Paris
2	2	Martin	Paul	Marseille	4500	2	Marseille	Paris	10:30:00	12:30:00	Boeing 737	180	Marseille
2	2	Martin	Paul	Marseille	4500	12	Marseille	Paris	10:30:00	12:30:00	Boeing 737	180	Marseille
3	3	Dubois	Marie	Lyon	4800	3	Lyon	Toulouse	09:15:00	10:45:00	Airbus A380	500	Lyon
3	3	Dubois	Marie	Lyon	4800	13	Lyon	Toulouse	09:15:00	10:45:00	Airbus A380	500	Lyon
4	4	Lefebvre	Sophie	Toulouse	5200	4	Toulouse	Lyon	11:30:00	12:45:00	Boeing 747	400	Toulouse
4	4	Lefebvre	Sophie	Toulouse	5200	14	Toulouse	Lyon	11:30:00	12:45:00	Boeing 747	400	Toulouse
5	5	Moreau	Pierre	Nice	4900	5	Nice	Bordeaux	10:00:00	12:00:00	Embraer E190	100	Nice
5	5	Moreau	Pierre	Nice	4900	15	Nice	Bordeaux	10:00:00	12:00:00	Embraer E190	100	Nice
6	6	Garcia	Julie	Bordeaux	5100	6	Bordeaux	Nice	13:00:00	15:00:00	Airbus A330	250	Bordeaux

AVNUM

Les sous-requêtes/ requêtes imbriquées

Une autre manière de combiner des relations

Les sous-requêtes avec IN

Les numéros des pilotes de Paris :

```
select PLNUM
from   pilote
where  VILLE = 'Paris';
```

Les numéros et villes de départ de vols qui ont utilisé des avions

Situés à Paris:

```
select VOLNUM, VILLEDEP
from   vol
where  AVNUM in(3, 6, 7, 10);
```

Cas particulier avec la Liste (3,6,7,10)

```
select VOLNUM, VILLEDEP
from   vol
where  AVNUM in (select AVNUM
                 from   avion
                 where  LOCALISATION = 'Paris');
```

Cas plus général

Les sous-requêtes avec IN

```
select *
from   avion
where  AVNUM in
       (select AVNUM
        from   vol
        where  VOLNUM in
               (select VOLNUM
                from   vol
                where  PLNUM in
                       (select PLNUM
                        from   pilote
                        where  VILLE='Paris')));
```

- les pilotes de Paris
- les vols pilotés par des pilotes de Paris
- les avions utilisés dans les vols pilotés par des pilotes de Paris
- les détails des avions utilisés dans les vols pilotés par des pilotes de Paris

Les sous-requêtes – conditions d'association

Remarque : symétrie des associations

```
select *
from   vol
where  PLNUM in (select PLNUM
                 from   pilote
                 where  VILLE = 'Paris');
```

```
select *
from   pilote
where  PLNUM in (select PLNUM
                 from   vol
                 where  VILLEDEP = 'Paris');
```

Les sous-requêtes – Références multiples

Références multiples à une même table

Sous-requêtes corrélées

```
select PLNUM, PLNOM, PLPRENOM, VILLE
from   pilote as P
where  SALAIRE > (select avg(SALAIRE)
                  from   pilote
                  where  VILLE = P.VILLE);
```

« P" est un alias de table

« P » est une variable qui à chaque instant référence la ligne courante de pilote dans la requête externe (on suppose qu'on examine successivement les lignes de pilote)

Les sous-requêtes – Références multiples

Condition d'association quantifiée : ***recherche des pilotes ayant effectué au moins 3 vols***

```
select PLNOM, PLPRENOM
from   pilote P
where  (select count(*)
        from vol
        where PLNUM = P.PLNUM ) >= 3;
```

Les quantificateurs ensemblistes – EXISTS, NOT EXISTS

<u>exists</u> et <u>not exists</u>

le prédicat exists(E), où E est une sous-requête,
est *vrai* si l'ensemble désigné par E est *non vide*

<u>Exemple</u> **:** quels sont les avions pour lesquels il existe au moins un vol ?

```
select AVNUM, AVNOM
from   avion as A
where  exists (select *
               from   vol
               where  AVNUM = A.AVNUM);
```

*On peut aussi utiliser une valeur par défaut pour ne pas garder en mémoire trop de données : select **1** from vol…*

le prédicat not exists(E), est vrai si l'ensemble désigné par E est vide

Equivalence requêtes imbriquées et jointures

Exemple : quels sont les avions pour lesquels il existe au moins un vol ?

Sinon un avion qui fait plusieurs vols apparaîtra plusieurs fois

```
select AVNUM, AVNOM
from   avion as A
where  exists (select *
               from   vol
               where  AVNUM = A.AVNUM);
```

Requête imbriquée avec **EXISTS**

```
select AVNUM, AVNOM
from avion as A
where AVNUM IN (
                select AVNUM
                from vol
                );
```

Requête imbriquée avec **IN**

```
select distinct AVNUM, AVNOM
from avion as A
join vol as V
on A.AVNUM = V.AVNUM;
```

Jointure interne

Ces trois requêtes donnent ***le même résultat.***
Pourquoi avoir différentes manières de l'écrire ? *Certaines sont* ***plus intuitives*** *dans certains contextes.*

Les quantificateurs ensemblistes – ALL, ANY

« Les pilotes dont le salaire est supérieur à tous les salaires des pilotes travaillant dans la ville de Paris »

```
SELECT PLNUM, PLNOM, PLPRENOM, SALAIRE
FROM PILOTE
WHERE SALAIRE > ALL (SELECT SALAIRE FROM PILOTE WHERE VILLE = 'Paris');
```

PLNUM	PLNOM	PLPRENOM	SALAIRE
14	Lefebvre	Sophie	5200
16	Garcia	Julie	5100
18	Leroy	Céline	5300
20	Sanchez	Lucie	5400

1. La sous-requête **(SELECT SALAIRE FROM PILOTE WHERE VILLE = 'Paris')** récupère tous les salaires des pilotes travaillant à Paris.
2. L'opérateur '**ALL**' compare le salaire de chaque pilote avec tous les salaires récupérés de la sous-requête.
3. La requête principale sélectionne les pilotes dont le salaire est supérieur à tous les salaires des pilotes travaillant à Paris.

variante

```
SELECT PLNUM, PLNOM, PLPRENOM, SALAIRE
FROM PILOTE
WHERE SALAIRE > (SELECT MAX(SALAIRE) FROM PILOTE WHERE VILLE = 'Paris');
```

Les quantificateurs ensemblistes – ALL, ANY

« Les vols dont l'heure de départ est antérieure à au moins un vol partant de la ville de Lyon.»

VOLNUM	VILLEDEP	VILLEARR	HEUREDEP	HEUREARR	AVNUM	PLNUM
1	Paris	Marseille	08:00:00	10:00:00	1	1
11	Paris	Marseille	08:00:00	10:00:00	1	1

```
SELECT *
FROM VOL
WHERE HEUREDEP < ANY (SELECT HEUREDEP FROM VOL WHERE VILLEDEP = 'Lyon');
```

1. La sous-requête **(SELECT HEUREDEP FROM VOL WHERE VILLEDEP = 'Lyon')** récupère toutes les heures de départ des vols partant de Lyon.
2. L'opérateur '**ANY**' compare l'heure de départ de chaque vol avec au moins l'une des heures de départ récupérées de la sous-requête.
3. La requête principale sélectionne tous les vols dont l'heure de départ est antérieure à au moins un vol partant de Lyon.

variante

```
SELECT *
FROM VOL
WHERE HEUREDEP < (SELECT MAX(HEUREDEP) FROM VOL WHERE VILLEDEP = Lyon');
```

Défi 1 : Tester vos connaissances !

Pour terminer, défi 1:
que signifie la requête suivante ?

```
SELECT DISTINCT P.PLNUM, P.PLNOM, P.PLPRENOM
FROM PILOTE P
WHERE P.PLNUM IN (
    SELECT DISTINCT V.PLNUM
    FROM VOL V
    WHERE V.AVNUM IN (
        SELECT AVNUM
        FROM AVION
        WHERE CAPACITE > (SELECT AVG(CAPACITE) FROM AVION)
    )
);
```

Défi 1 : Tester vos connaissances !

Pour terminer, défi 1:
que signifie la requête suivante ?

```sql
SELECT DISTINCT P.PLNUM, P.PLNOM, P.PLPRENOM
FROM PILOTE P
WHERE P.PLNUM IN (
    SELECT DISTINCT V.PLNUM
    FROM VOL V
    WHERE V.AVNUM IN (
        SELECT AVNUM
        FROM AVION
        WHERE CAPACITE > (SELECT AVG(CAPACITE) FROM AVION)
    )
);
```

Réponse : Affiche les informations des pilotes qui sont affectés à des vols dont la capacité de l'avion est supérieure à la moyenne de toutes les capacités d'avion.

1. Nous sélectionnons les pilotes distincts (car un pilote peut être affecté à plusieurs vols) de la table PILOTE (alias P).
2. Dans la sous-requête la plus interne, nous sélectionnons les numéros d'avion des avions dont la capacité est supérieure à la moyenne de toutes les capacités d'avion.
3. Dans la sous-requête intermédiaire, nous sélectionnons les numéros de pilote des pilotes affectés à des vols où l'avion a une capacité supérieure à la moyenne.
4. Finalement, dans la requête principale, nous sélectionnons les détails distincts des pilotes dont le numéro de pilote est présent dans les résultats de la sous-requête intermédiaire.

Défi 2 : Tester vos connaissances !

Pour terminer, défi 2:
Réécrire la requête du défi 1 en utilisant une jointure

Requête : Afficher les informations des pilotes qui sont affectés à des vols dont la capacité de l'avion est supérieure à la moyenne de toutes les capacités d'avion.

```sql
SELECT DISTINCT P.PLNUM, P.PLNOM, P.PLPRENOM
FROM PILOTE P
WHERE P.PLNUM IN (
    SELECT DISTINCT V.PLNUM
    FROM VOL V
    WHERE V.AVNUM IN (
        SELECT AVNUM
        FROM AVION
        WHERE CAPACITE > (SELECT AVG(CAPACITE) FROM AVION)
    )
);
```

Défi 2 : Tester vos connaissances !

Pour terminer, défi 2:
Réécrire la requête du défi 1 en utilisant une jointure

Requête : Afficher les informations des pilotes qui sont affectés à des vols dont la capacité de l'avion est supérieure à la moyenne de toutes les capacités d'avion.

```sql
SELECT DISTINCT P.PLNUM, P.PLNOM, P.PLPRENOM
FROM PILOTE P
WHERE P.PLNUM IN (
    SELECT DISTINCT V.PLNUM
    FROM VOL V
    WHERE V.AVNUM IN (
        SELECT AVNUM
        FROM AVION
        WHERE CAPACITE > (SELECT AVG(CAPACITE) FROM AVION)
    )
);
```

→

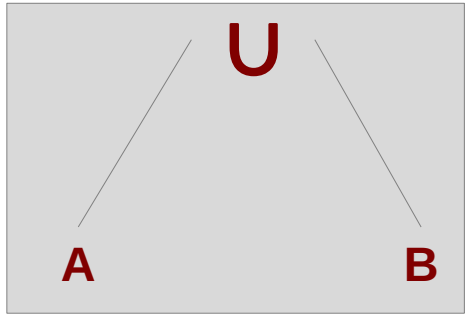
```sql
SELECT DISTINCT P.PLNUM, P.PLNOM, P.PLPRENOM
FROM PILOTE P
JOIN VOL V ON P.PLNUM = V.PLNUM
JOIN AVION A ON V.AVNUM = A.AVNUM
WHERE A.CAPACITE > (SELECT AVG(CAPACITE) FROM AVION);
```

Partie 4 : Opérateurs ensemblistes

∪ Union

Rq : R1 et R2 ont le même schéma / même nombre d'attributs, mêmes domaines

R3 = R1 ∪ R2 est composée des tuples qui apparaissent dans R1 ou bien dans R2.



arbre relationnel

R 1	att1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25

∪

R 2	att 1	att 2	att 3	att4
	1	2	2	2
	3	3	3	20
	5	3	3	25

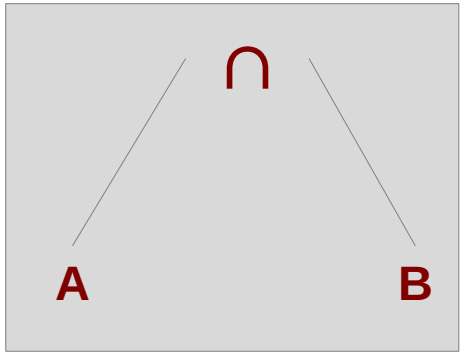
=

R 3	att1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25
	3	3	3	20

∩ Intersection

Rq : R1 et R2 ont le même schéma / même nombre d'attributs, mêmes domaines

Conserver les tuples qui appartiennent à la fois à R1 et R2.



arbre relationnel

R 1	att 1	att2	att3	att4
	1	2	2	2
	3	3	3	25
	5	3	3	25

∩

R 2	att1	att2	att3	att4
	1	2	2	2
	3	3	3	20
	5	3	3	25

=

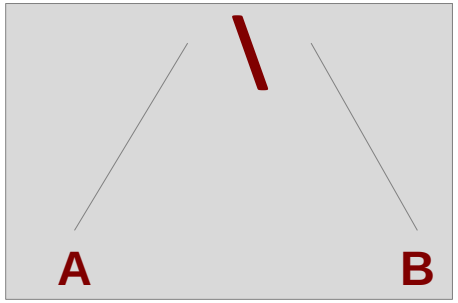
R 3	att1	att2	att3	att4
	1	2	2	2
	5	3	3	25

\ Différence

Rq : R1 et R2 ont le même schéma / même nombre d'attributs, mêmes domaines

Conserver les tuples de R1 seulement s'ils n'existent pas dans R2.

Attention ! La différence n'est pas commutative.



arbre relationnel

R 1	att 1	att2	att 3	att 4
	1	2	2	2
	3	3	3	25
	5	3	3	25

\

R 2	att 1	att 2	att 3	att 4
	1	2	2	2
	3	3	3	20
	5	3	3	25

=

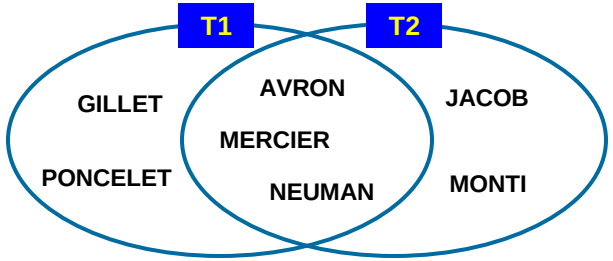
R 3	att 1	att 2	att 3	att 4
	3	3	3	25

SQL : Opérations ensemblistes

Opérateurs ensemblistes entre 2 tables sans doublons

T1
NOM
GILLET
AVRON
MERCIER
PONCELET
NEUMAN

T2
NOM
MONTI
NEUMAN
JACOB
MERCIER
AVRON



```
select NOM
from   T1
union
select NOM
from   T2
```

NOM
GILLET
AVRON
MERCIER
PONCELET
NEUMAN
MONTI
JACOB

```
select NOM
from   T1
intersect
select NOM
from   T2
```

NOM
AVRON
MERCIER
NEUMAN

```
select NOM
from   T1
except
select NOM
from   T2
```

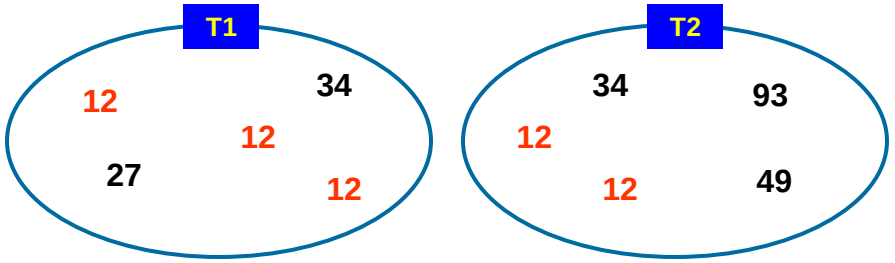
NOM
GILLET
PONCELET

SQL : Opérations ensemblistes

Opérateurs ensemblistes entre 2 tables avec doublons

T1
NUM
12
34
27
12
12

T2
NUM
93
12
34
12
49



```
select NUM
from   T1
union
select NUM
from   T2
```

NUM
12
34
27
93
49

```
select NUM
from   T1
intersect
select NUM
from   T2
```

NUM
34
12

```
select NUM
from   T1
except
select NUM
from   T2
```

NUM
27

```
select NUM
from   T2
except
select NUM
from   T1
```

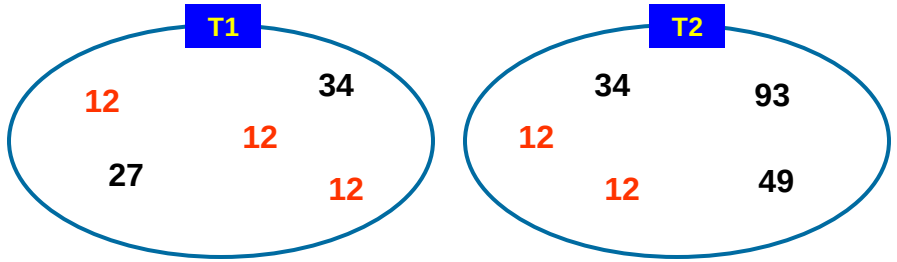
NUM
93
49

SQL : Opérations ensemblistes – les multi-ensembles

Opérateurs multi-ensemblistes entre 2 tables avec doublons

T1
NUM
12
34
27
12
12

T2
NUM
93
12
34
12
49




```
select NUM
from   T1
union all
select NUM
from   T2
```

NUM
12
12
12
34
27
12
12
34
93
49

```
select NUM
from   T1
intersect all
select NUM
from   T2
```

NUM
34
12
12

```
select NUM
from   T1
except all
select NUM
from   T2
```

NUM
12
27

```
select NUM
from   T2
except all
select NUM
from   T1
```

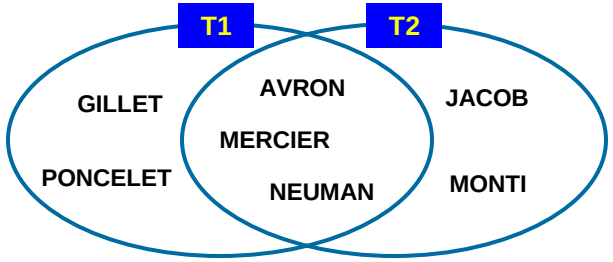
NUM
93
49

SQL : Opérations ensemblistes – Expressions complexes

Les opérateurs peuvent être combinés (ex. *différence symétrique*)

T1
NOM
GILLET
AVRON
MERCIER
PONCELET
NEUMAN

T2
NOM
MONTI
NEUMAN
JACOB
MERCIER
AVRON



```
(select NOM
 from  T1
 except
 select NOM
 from  T2)
union
(select NOM
 from  T2
 except
 select NOM
 from  T1)
```

⇒

NOM
GILLET
PONCELET
JACOB
MONTI

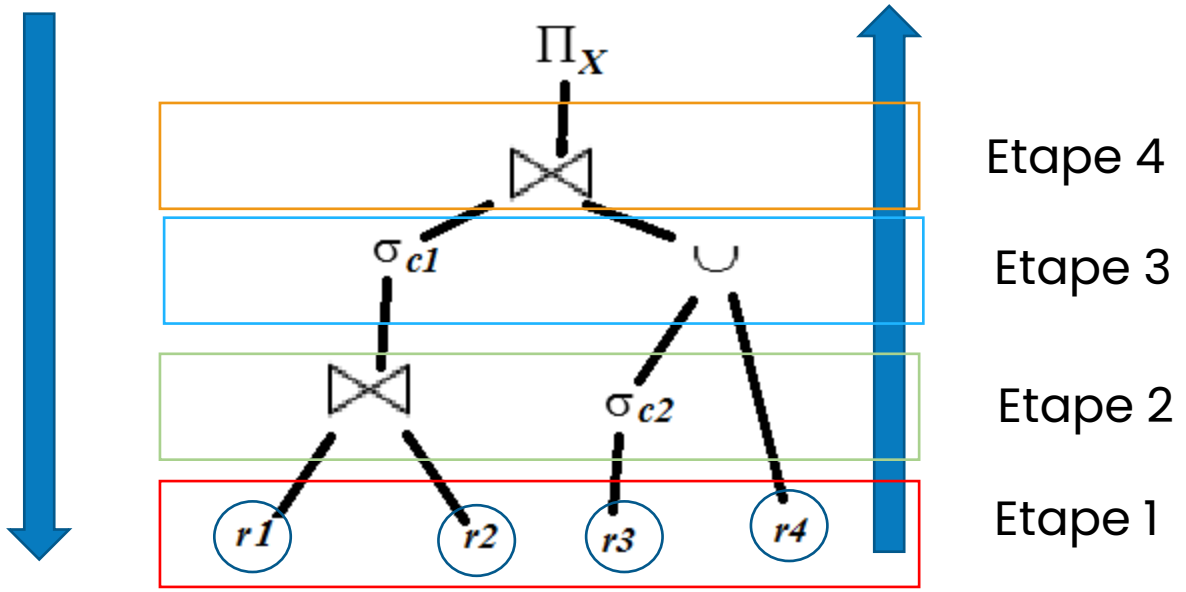
Partie 5 : Arbre relationnel et plan logique

Algèbre relationnelle : Représentation graphique

Un programme d'opérations de l'algèbre relationnelle peut être représenté par un arbre :

- Les feuilles de l'arbre correspondent aux relations
- Les nœuds de l'arbre correspondent aux opérateurs algébriques

$$Q = \pi_X(\sigma_{C_1}(r_1 \bowtie r_2) \bowtie (\sigma_{C_2}(r_3) \cup r_4))$$



Pourquoi connaître l'algèbre relationnelle ?

La décomposition des requêtes SQL en opérations logiques, celles de l'algèbre relationnelle, est faite par le SGBD avant l'exécution. Il établit un **plan logique**, qui est l'optimisation de l'arbre relationnel obtenu.

Exemple : les noms des pilotes qui font des vols qui arrivent à Paris

```
SELECT P.PLNOM
FROM pilote P
JOIN vol V ON P.PLNUM = V.PLNUM
WHERE V.VILLE_ARRIVEE = 'Paris';
```

```
      π PLNOM
         |
σ VILLE_ARRIVEE='Paris'
         |
      ⋈ P.PLNUM = V.PLNUM
      /        \
 pilote P      vol V
```

La jointure est coûteuse : elle fait garder en mémoire beaucoup de tuples

Arbre relationnel initial

```
      π PLNOM
         |
      ⋈ P.PLNUM = V.PLNUM
      /        \
 pilote P      σ VILLE_ARRIVEE='Paris'
                  |
                vol V
```

La jointure se fait sur une relation « plus petite »

Arbre relationnel optimisé = **plan logique**

Quelques règles pour l'optimisation

- **Pousser les sélections** : le plus tôt possible *(le plus bas dans l'arbre relationnel)*, soit avant les jointures si elles ne concernent qu'une seule table

- **Pousser les projections** : le plus tôt possible *(le plus bas dans l'arbre relationnel)*, en faisant attention de conserver tous les attributs nécessaires pour les opérations qui s'appliquent après

- Utiliser l'opération spécifique de **jointure** (qui ne génère pas toutes les combinaisons) plutôt qu'un produit cartésien + sélection

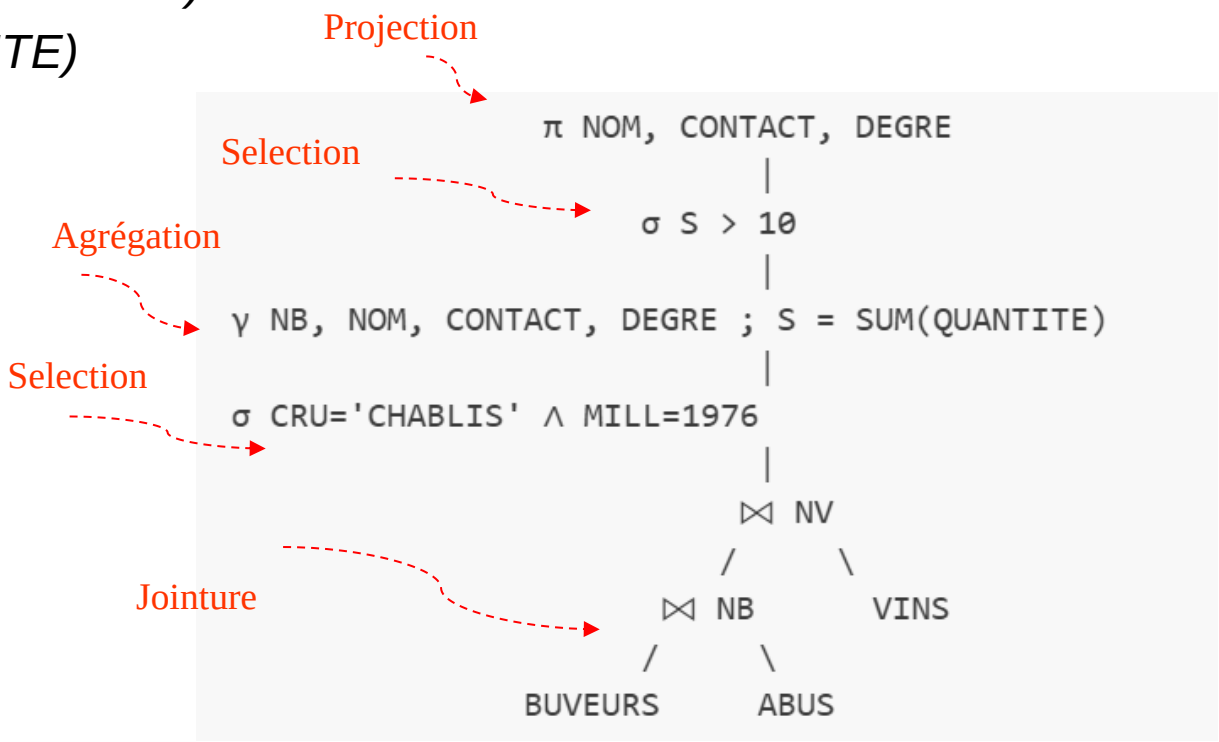
C'est ce que fait automatiquement le SGBD, mais en fonction de celui choisi et de comment certaines requêtes sont écrites en SQL, les choix d'optimisation peuvent être différents.

Création d'un arbre relationnel

Exemple : donner les noms et contacts des buveurs ayant bu plus de 10 bouteilles au total de CHABLIS 1976 et le degré de ce vin.

- ***BUVEURS(NB**, NOM, PRENOM, CONTACT)*
- ***VINS(NV**, CRU, MILL, QUALITE, DEGRE)*
- ***ABUS(#NB,# NV, Date**, QUANTITE)*

```
SELECT b.NOM, b.CONTACT, v.DEGRE
FROM BUVEURS b
JOIN ABUS a ON b.NB = a.NB
JOIN VINS v ON a.NV = v.NV
WHERE v.CRU = 'CHABLIS' AND v.MILL = 1976
GROUP BY b.NB, b.NOM, b.CONTACT, v.DEGRE
HAVING SUM(a.QUANTITE) > 10;
```



Peut-on l'optimiser ?

FIN

01